

Project Workflow: Context-Aware Hybrid Engine

This document outlines the current data flow of the **AI Contract Risk Platform**. The system operates on a **Hybrid Architecture** where Rule-Based logic (Regex) takes precedence for speed and determinism, while the AI Model (LegalBERT) is used for context when rules are insufficient.

1. High-Level Pipeline

⚠ Failed to render Mermaid diagram: Parse error on line 6

graph TD

```
A[Upload Contract] --> B[Text Extraction]
B --> C[Segmentation]
C --> D{Classification Service}
D -- Strong Rule --> E[Rule Result]
D -- Weak Rule --> F[Contextual Classifier (LegalBERT)]
F --> G[Merged Class Result]
E --> G
G --> H[Risk Service (Signal Engine)]
H --> I{Contract Service (Hybrid Gate)}
I -- High Risk confirmed --> J[LLM Enrichment (Explanation)]
I -- Low/Med/Info --> K[Database]
J --> K
```

2. Detailed Process Flow

Step 1: Ingestion (

contract_service.py)

- **Input:** User uploads a PDF or DOCX file.
- **Action:** File is saved locally.
- **Service:** text_extractor extracts raw text.
- **Data Model:** A

Contract record is created in SQLite.

Step 2: Segmentation (

segmentation_service.py)

- **Action:** The raw text is split into logical **Clauses**.
- **Logic:** Uses regex to identify Article numbers (1., Section 2.1), line breaks, and indentation.
- **Output:** A list of segments (dictionaries with text and metadata).

Step 3: Classification (

classification_service.py)

Determines "What is this clause?" (e.g., Indemnity, Termination)

The Hybrid Logic:

1. **Rule Check (Stage 1):** Runs regex patterns (RULE_KEYWORDS).
 - If rule_score ≥ 0.75 (Strong Match): **STOP**. Use Rule result. (Fast, Deterministic).
2. **Contextual Fallback (Stage 2):** If Rules are weak (< 0.75):
 - Call

ContextualClassifier (LegalBERT).

- If Model Validity > 0.60 : Use Model result.
 - Else: Fallback to weak rule or "Unknown".
3. **Title Boost:** If clause heading matches keywords (e.g., "Liability"), boost score.
 4. **Exclusion:** If clause looks like a Definition or Signature, mark as NON_RISK.

Step 4: Risk Analysis (

risk_service.py)

Determines "How dangerous is this?"

- **Input:** Clause Text + Clause Type.
- **Signal Engine:** Scans for specific **Risk Signals** using regex (independent of Step 3).
 - *Example:* "Unlimited Liability", "Unilateral Termination", "Arbitration".
- **Snippets:** Extracts exact phrases (e.g., "shall be forfeited") for evidence.
- **Scoring:** Base Score (0-10) calculated from signals + modifiers (e.g., "Mutual" lowers risk).

Step 5: The Hybrid Gate (

contract_service.py)

Combines Classification risk_level (AI) with Risk Analysis risk_score (Rules).

Critical Logic (Lines 108-117):

- **Strict High Risk Gate:** A clause is upgraded to **High Risk** by the AI Model **ONLY IF**:
 1. Model predicts High Risk with **Confidence ≥ 0.80** .
 2. **AND** risk_service found at least one risk signal (score > 0).
- *Why?* To prevent the "Black Box" AI from flagging safe clauses as High Risk without pointing to a specific risky phrase (hallucination guardrail).

Step 6: Enrichment (

llm_service.py)

- **Trigger:** Only for confirmed **High Risk** clauses.
- **Action:** Calls LLM (Gemini/OpenAI) to generate:
 - **Explanation:** Plain English summary of why it's risky.
 - **Rewrite:** A safer suggestions.

Step 7: Storage & Summary

- **Database:** Clause records saved with clause_type, risk_score, risk_level.
- **Contract Summary:**
 - Aggregates clause scores.
 - Determines overall_risk_level (Low/Medium/High).
 - Identifies "Top 3 Risk Drivers".

Key File Locations

- **Orchestrator:**

app/services/contract_service.py

- **Classifier:**

app/services/classification_service.py

- **Rule Engine:**

app/services/risk_service.py

- **Models:** app/domain/models.py